PONTIFÍCIA UNIVERSIDADE CATÓLICA DE GOIÁS
COORDENAÇÃO DE EDUCAÇÃO A DISTÂNCIA – CEAD
ESCOLA POLITÉCNICA E DE ARTES

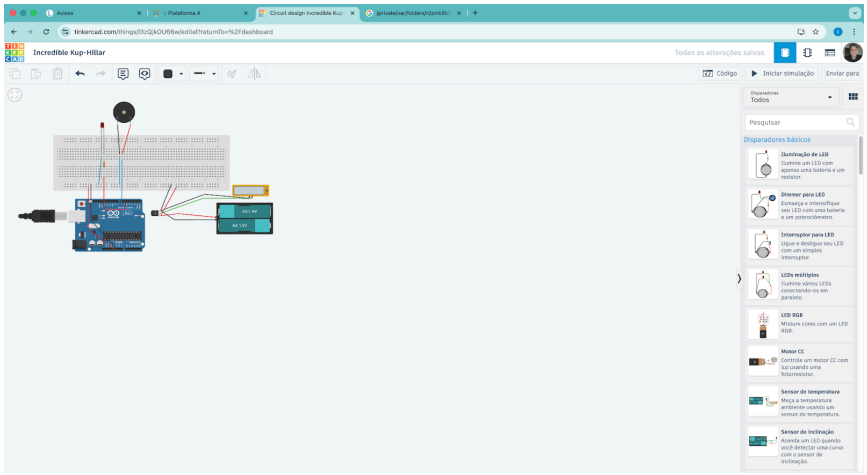
PROPOSTA – PROJETO INTEGRADOR V – B
SISTEMA DE MONITORAMENTO DE AMBIENTES INTELIGENTES

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Prof. Thalles Bruno G. N. dos Santos
Goiânia – 2026

Aluno : Elione Felix de Sá

1. Esquema Ilustrativo do Protótipo Arduino



link de acesso ao thinkcard:

https://www.tinkercad.com/things/l3zQjkOU66w/editel?returnTo=%2Fdashboard

Descrição dos funcionamentos, esquemas e componentes utilizados (Arduino, sensores, LEDs,resistores, etc.).

O protótipo desenvolvido consiste em um sistema de monitoramento ambiental utilizando uma placa Arduino UNO, sensores de temperatura e umidade, sensor de luminosidade (LDR), resistores e LEDs para sinalização. O objetivo do sistema é coletar dados do ambiente em tempo real e enviá-los para o computador, onde podem ser processados por um módulo em Java e exibidos em uma interface gráfica.

Funcionamento geral do sistema

O Arduino UNO atua como unidade central de controle do circuito. Ele recebe os dados dos sensores conectados à placa, realiza a leitura dessas informações por meio de suas portas digitais e analógicas e envia os valores pela comunicação serial para o computador. O sensor de temperatura é responsável por medir a temperatura e a umidade do ambiente. Esse sensor possui três pinos principais:

alimentação (5V), terra (GND) e sinal de dados. O pino de dados é conectado a uma porta digital do Arduino, permitindo que o microcontrolador leia os valores medidos pelo sensor.

O sensor de luminosidade LDR (Light Dependent Resistor) é utilizado para medir a intensidade de luz do ambiente. O LDR funciona variando sua resistência conforme a quantidade de luz recebida. Para que o Arduino consiga ler esse valor, o LDR é ligado em conjunto com um resistor formando um divisor de tensão, e o sinal resultante é conectado a uma porta analógica do Arduino.

Os resistores são utilizados para proteger os componentes e garantir o funcionamento correto do circuito. No caso do LDR, o resistor permite transformar a variação de resistência

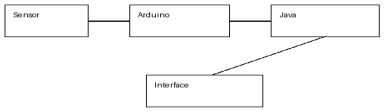
Os LEDs são utilizados como indicadores visuais do estado do sistema. Por exemplo, um LED pode acender quando a temperatura estiver acima de um valor definido, ou quando a luminosidade estiver baixa. Cada LED é conectado a um pino digital do Arduino com um resistor em série para controle da corrente.

Após realizar a leitura dos sensores, o Arduino envia os dados pela porta serial utilizando comunicação USB. Esses dados são recebidos por um programa desenvolvido em Java, que interpreta as informações e as apresenta em uma interface gráfica, permitindo a visualização em tempo real das condições do ambiente.

Dessa forma, o sistema integra hardware e software, utilizando conceitos de Internet das Coisas (IoT), permitindo monitoramento contínuo, visualização de dados e

possibilidade de futuras automações, como controle de iluminação ou ventilação automática.

2. Diagramas ilustrativos do protótipo.



3. código Arduino utilizados no projeto.

```
#include "DHT.h"
#define DHTPIN 2
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

void setup() {
 Serial.begin(9600);
 dht.begin();
}

void loop() {
 delay(2000);

 float h = dht.readHumidity();
 float t = dht.readTemperature();

  if (isnan(h) || isnan(t)) {
   Serial.println("Erro ao ler o sensor!");
   return;
 }

 Serial.print("Umidade: ");
 Serial.print(h);
 Serial.print("% | Temperatura: ");
 Serial.print(t);
 Serial.println("°C");
 }
```

4. Código fonte completo do módulo Java.

```
import java.util.Random;
```

```
public class SensorDHTSimulado {

    static int TempMax = 35;
    static int HumiMin = 50;

    public static void main(String[] args) throws InterruptedException {

        Random random = new Random();

        while (true) {

            // Simulação dos valores do sensor
            int temperatura = 20 + random.nextInt(20);
            int humidade = 30 + random.nextInt(50);

            System.out.println("Humidity: " + humidity + " %");
            System.out.println("Temperature: " + temperature + " °C");

            if (temperatura > TempMax) {
                System.out.println("   Temperature alta! LED ON");
                System.out.println("Buzzer: BIP");
                Thread.sleep(500);
                System.out.println("Buzzer OFF");
                Thread.sleep(10000);
                System.out.println("LED OFF");
            }

            if (humidade < HumiMin) {
                System.out.println("⚠  humidity baixa! LED ON");

                for (int i = 0; i < 3; i++) {
                    System.out.println("Buzzer: BIP");
                    Thread.sleep(500);
                    System.out.println("Buzzer OFF");
                    Thread.sleep(500);
                }

                Thread.sleep(5000);
                System.out.println("LED OFF");
            }

            System.out.println("-----------------------");
            Thread.sleep(2000);
        }
    }
}
```

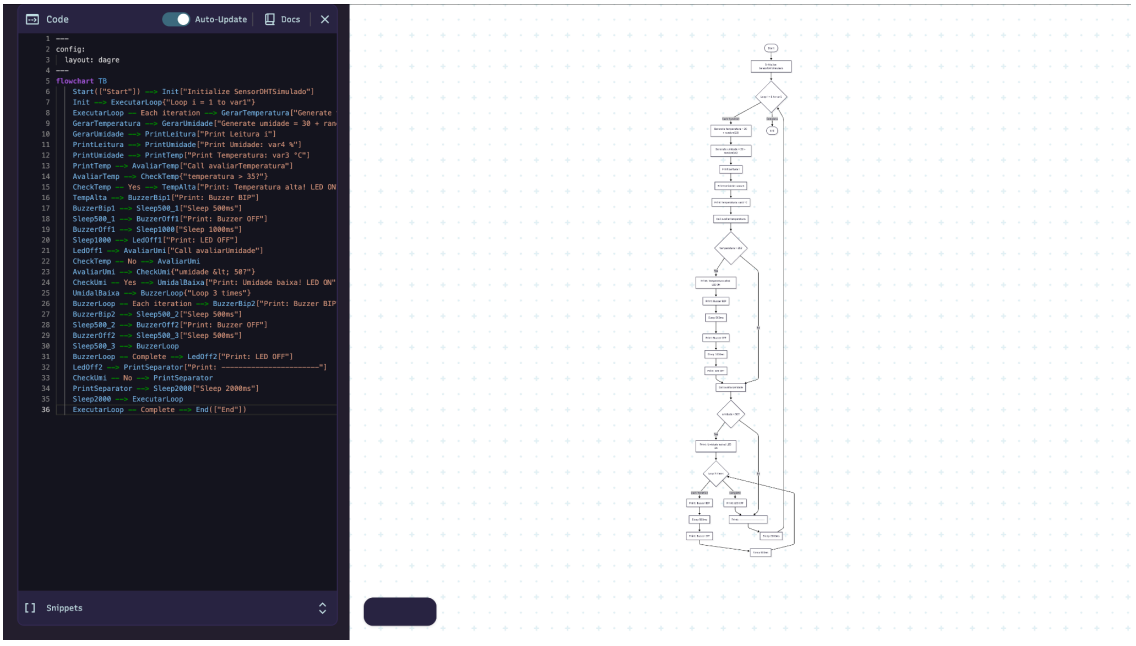
5. Descrição do funcionamento do módulo Java:

```
import java.util.Random;

/**
 * Classe que simula o funcionamento de um sensor DHT11.
 *
 * <p>Este programa gera valores aleatórios de temperatura e umidade,
 * simulando a leitura de um sensor físico. Com base nesses valores,
 * o sistema executa ações simuladas como ativação de LED e buzzer.</p>
 *
 * <h2>Regras de funcionamento:</h2>
 * <ul>
 *   <li>Se a temperatura for maior que {@code TempMax}, um alerta é acionado</li>
 *   <li>Se a umidade for menor que {@code HumiMin}, um alerta é acionado</li>
 * </ul>
 *
```

```
 * <h2>Comportamentos simulados:</h2>
 * <ul>
 *   <li>LED ligado/desligado</li>
 *   <li>Buzzer emitindo sons (bips)</li>
 * </ul>
 *
 * <p>O sistema roda continuamente em loop, simulando um dispositivo IoT.</p>
 *
 * @author Elione de Sa
 * @version 1.0
 */
public class SensorDHTSimulado {

    /** Temperatura máxima permitida (°C) */
    static int TempMax = 35;

    /** Umidade mínima permitida (%) */
    static int HumiMin = 50;

    /**
     * Método principal que executa a simulação do sensor.
     *
     * @param args argumentos da linha de comando (não utilizados)
     * @throws InterruptedException caso ocorra interrupção no delay
     */
    public static void main(String[] args) throws InterruptedException {

        Random random = new Random();

        while (true) {

            // Simulação dos valores do sensor
            int temperatura = 20 + random.nextInt(20);
            int humidade = 30 + random.nextInt(50);

              System.out.println("Humidade: " + humidade + " %");
            System.out.println("Temperatura: " + temperatura + " °C");

            if (temperatura > TempMax) {
                System.out.println("   temperature alta! LED ON");
                System.out.println("Buzzer: BIP");
                Thread.sleep(500);
                System.out.println("Buzzer OFF");
                Thread.sleep(10000);
                System.out.println("LED OFF");
            }

              if (humidade < HumiMin) {
                  System.out.println("   humidity baixa! LED ON");

                for (int i = 0; i < 3; i++) {
                    System.out.println("Buzzer: BIP");
```

```
                Thread.sleep(500);
                System.out.println("Buzzer OFF");
                Thread.sleep(500);
            }

            Thread.sleep(5000);
            System.out.println("LED OFF");
        }

        System.out.println("----------------------");
        Thread.sleep(2000);
    }
  }
}
```

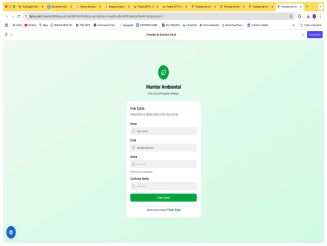
6. Diagrama de classes UML (print) e o link da plataforma utilizada.

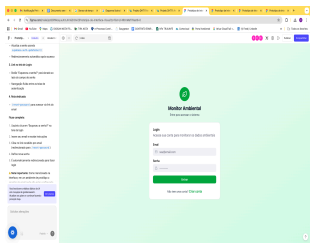


https://mermaid.ai/app/projects/5d621a28-9a48-4161-bc09-d2249939019d/diagrams/73b4406d-de13-442d-82c0-d1f474389cd5/version/v0.1/edit

7. Capturas de tela da interface.

CADASTRO LOGIN MENU





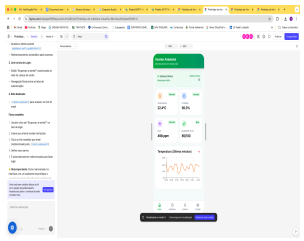
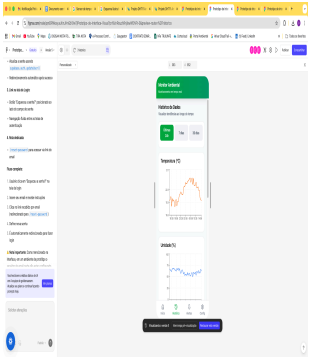
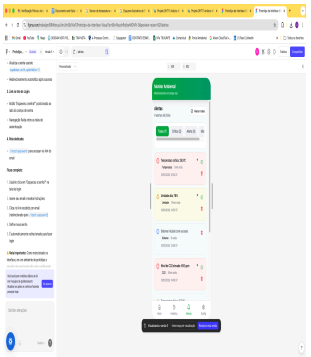
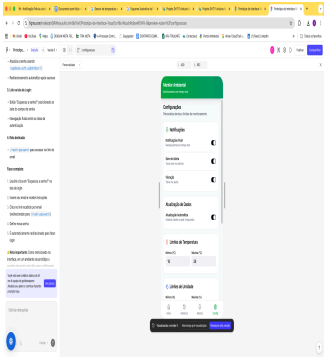


GRÁFICO ALERTAS CONFIGURAÇÕES







https://www.figma.com/make/qmEIRWoxyuuXmJHnGb1XwT/Prototipo-de-Interface-Visual?fullscreen=1&t=h0dd6lItXUTzYFqc-1

8. Breve explicação das Funcionalidades da interface (console)

A interface do sistema é baseada no **terminal/console**, onde todas as informações são exibidas em tempo real.

Exibição de dados

- Mostra continuamente os valores de:

  Temperatura (°C)

  Umidade (%)

Monitoramento automático

O sistema analisa os valores gerados e compara com limites definidos:

Temperatura máxima (`TempMax`)

Umidade mínima (`HumiMin`)

Alertas visuais (simulados)

Quando ocorre uma condição crítica, Exibe mensagens como:

`(Temperatura alta! LED ON)(Umidade baixa! LED ON)`

Alertas sonoros (simulados)

Simula o funcionamento de um buzzer:

- 1 bip → temperatura alta
- 3 bips → umidade baixa

Atualização contínua

Os dados são atualizados em loop infinito

Pequenos intervalos (`delay`) simulam o tempo real de leitura

Separação visual

Linha separadora (`----------`) para organizar a saída no console

Resumo:

A interface permite **acompanhar, em tempo real**, os dados do sensor simulado e identificar rapidamente situações de alerta através de mensagens visuais e sonoras.

Referências:

As referências foram utilizadas para fundamentar o desenvolvimento deste trabalho, fornecendo suporte teórico e técnico sobre o funcionamento do sensor de temperatura e umidade, bem como sobre a implementação da linguagem Java.A documentação oficial da contribuiu para a compreensão do funcionamento do sensor em sistemas embarcados, enquanto a biblioteca DHT e materiais técnicos auxiliaram na lógica de leitura dos dados. Já a documentação da linguagem Java foi utilizada como base para a implementação da simulação do sistema.

Dessa forma, as fontes consultadas garantem maior confiabilidade, embasamento técnico e consistência ao projeto desenvolvido.

Documentação oficial do Java. Disponível em:
https://docs.oracle.com/javase/8/docs/
Classe `Random` – Java API. Disponível em:
https://docs.oracle.com/javase/8/docs/api/java/util/Random.html
– Documentação oficial. Disponível em: https://www.arduino.cc/
Biblioteca DHT (Adafruit). Disponível em:
https://github.com/adafruit/DHT-sensor-library
Tutoriais sobre sensor DHT11 – UsinaInfo. Disponível em:
https://www.usinainfo.com.br/blog/